

Amazon Ads Automation Bot — Architecture V1 FINALE (VALIDÉE)

Version : 1.2 (Finale)

Date : 1er Février 2026

Statut : FIGÉE POUR IMPLÉMENTATION

Changements vs Version Précédente

#	Problème identifié	Correction appliquée
1	entity_id UUID polymorphe	→ <code>entity_key</code> TEXT (format: <code>type:amazon_id</code>)
2	UNIQUE trop restrictif sur marketplace_profiles	→ <code>UNIQUE(ad_account_id, profile_id)</code>
3	Redondance ad_account_id dans portfolios	→ Supprimé, on passe par profile_id
4	search_terms sans contexte campaign/ad_group	→ Ajout campaign_id, ad_group_id
5	Pas de tracking 60 jours search terms	→ Ajout last_search_terms_sync dans profiles
6	hourly_budget_snapshots trop complexe	→ Déplacé en V1.1
7	KPIs stockés avec risque div/0	→ KPIs calculés à la volée dans vues
8	users.password_hash inutile V1	→ Supprimé
9	access_token en DB	→ Cache mémoire uniquement
10	action_log avec UUID polymorphe	→ <code>entity_key</code> TEXT
11	Pas de stratégie idempotence reports	→ UNIQUE sur report_jobs + UPSERT
12	Pas de versioning règles	→ <code>rule_snapshot</code> JSONB dans recommendations

- | | | |
|---|---------------------------------|--|
| 1 | Doublons de search terms | → query_norm + hash sur query_norm |
| 3 | (casse/espaces) | |
| 1 | Collision métriques search_term | → entity_key search_term = |
| 4 | (hash non unique global) | search_term:{amazon_ad_group_id}:{query_hash} |
| 1 | report_jobs "completed" ambigu | → statuts ready/downloaded/ingested + |
| 5 | | timestamps |
-

1. Configuration V1 Figée

Hypothèses de départ (modifiables après)

marketplace_v1: FR

mode_default: recommend # Pas d'auto-exécution au début

acos_target_default: 40 # %

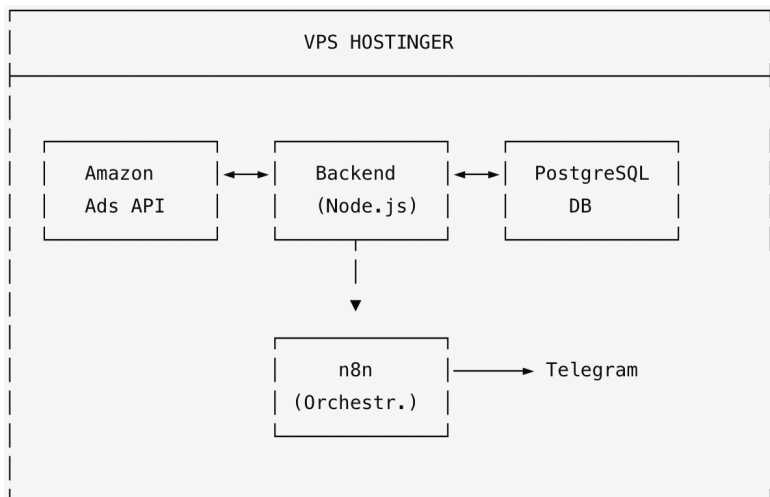
dry_run_initial: true # Passer à false après validation

min_clicks_decision: 15

min_spend_decision: 5.00 # EUR

2. Architecture Générale

2.1 Vue d'Ensemble



2.2 Principes

1. **n8n orchestre / Backend décide / DB stocke**
2. **Idempotence** : Chaque job est relançable (UPSERT partout)
3. **Fail-safe** : Erreur → log + alerte, pas de crash

4. **Audit complet** : Avant/après tracé

2.3 Convention **entity_key** (POLYMORPHE)

Pour tables polymorphes (metrics, logs), on utilise **entity_key** TEXT.

Format standard

{entity_type}:{amazon_id}

Exemples :

- **campaign:123456789**
- **ad_group:987654321**
- **keyword:456789123**
- **target:789123456**

Format search_term (IMPORTANT : contextuel)

Pour éviter collisions entre ad_groups :

search_term:{amazon_ad_group_id}:{query_hash}

Exemple :

- **search_term:987654321:1bc29b36f623ba82aaf6724fd3b16718**

Avantages :

- Pas de FK impossible sur table polymorphe
- Requêtes simples (split_part / LIKE)
- Compatible multi-entités
- **Aucune collision sur search terms**

3. Modèle de Données V1 FINAL

3.1 Migration 001 — Schema Initial (FINAL)

```
-- =====  
-- MIGRATION 001: Initial Schema (FINAL)  
-- =====
```

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
```

```
-- =====
```

-- CORE: Users & Workspaces (simplifié V1)

-- =====

```
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email VARCHAR(255) UNIQUE NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  is_active BOOLEAN DEFAULT true  
);
```

```
CREATE TABLE workspaces (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  name VARCHAR(255) NOT NULL,  
  settings JSONB DEFAULT '{  
    "default_acos_target": 40,  
    "notification_channels": ["telegram"],  
    "timezone": "Europe/Paris",  
    "auto_mode_enabled": false,  
    "dry_run": true  
}',  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  UNIQUE(user_id, name)  
);
```

-- =====

-- AMAZON ADS CONNECTION

-- =====

```
CREATE TABLE ad_accounts (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,  
  
  refresh_token_encrypted TEXT NOT NULL,  
  token_expires_at TIMESTAMP WITH TIME ZONE,  
  
  amazon_account_id VARCHAR(100),  
  account_name VARCHAR(255),  
  
  status VARCHAR(50) DEFAULT 'active'  
  CHECK (status IN ('active', 'paused', 'error', 'revoked')),  
  last_sync_at TIMESTAMP WITH TIME ZONE,  
  last_error TEXT,  
  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
```

```
);
```

```
CREATE TABLE marketplace_profiles (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  ad_account_id UUID NOT NULL REFERENCES ad_accounts(id) ON DELETE  
  CASCADE,
```

```
  profile_id BIGINT NOT NULL,  
  marketplace VARCHAR(10) NOT NULL  
    CHECK (marketplace IN ('FR','DE','US','UK','ES','IT','CA','AU')),  
  marketplace_id VARCHAR(50),  
  currency VARCHAR(3) NOT NULL  
    CHECK (currency IN ('EUR','USD','GBP','CAD','AUD')),
```

```
  is_active BOOLEAN DEFAULT true,  
  last_sync_at TIMESTAMP WITH TIME ZONE,  
  last_search_terms_sync_at TIMESTAMP WITH TIME ZONE,
```

```
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
```

```
  UNIQUE(ad_account_id, profile_id)
```

```
);
```

```
CREATE INDEX idx_profiles_marketplace  
  ON marketplace_profiles(ad_account_id, marketplace);
```

```
-- =====  
-- BOOKS CATALOG  
-- =====
```

```
CREATE TABLE books (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
```

```
  asin VARCHAR(20) NOT NULL,  
  marketplace VARCHAR(10) NOT NULL,  
  title VARCHAR(500),  
  author VARCHAR(255),
```

```
  kdp_id VARCHAR(100),  
  publication_date DATE,
```

```
  categories JSONB DEFAULT '[]',  
  tags JSONB DEFAULT '[]',
```

```
  acos_target DECIMAL(5,2),  
  daily_budget_target DECIMAL(10,2),
```

```

        created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
        updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

        UNIQUE(workspace_id, asin, marketplace)
    );

-- =====
-- AMAZON ADS STRUCTURE
-- =====

CREATE TABLE portfolios (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    profile_id UUID NOT NULL REFERENCES marketplace_profiles(id) ON DELETE
    CASCADE,

    amazon_portfolio_id BIGINT NOT NULL,
    name VARCHAR(255) NOT NULL,
    state VARCHAR(50) CHECK (state IN ('enabled','paused','archived')),

    budget_amount DECIMAL(10,2),
    budget_currency VARCHAR(3),
    budget_policy VARCHAR(50),

    last_synced_at TIMESTAMP WITH TIME ZONE,
    raw_data JSONB,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    UNIQUE(profile_id, amazon_portfolio_id)
);

CREATE TABLE campaigns (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    profile_id UUID NOT NULL REFERENCES marketplace_profiles(id) ON DELETE
    CASCADE,
    portfolio_id UUID REFERENCES portfolios(id) ON DELETE SET NULL,

    amazon_campaign_id BIGINT NOT NULL,

    name VARCHAR(500) NOT NULL,
    campaign_type VARCHAR(50) NOT NULL DEFAULT 'sponsoredProducts'
    CHECK (campaign_type IN
('sponsoredProducts','sponsoredBrands','sponsoredDisplay')),

    state VARCHAR(50) NOT NULL
    CHECK (state IN ('enabled','paused','archived')),
    targeting_type VARCHAR(50)

```

```

        CHECK (targeting_type IN ('manual','auto')),

        budget_type VARCHAR(50) DEFAULT 'daily',
        daily_budget DECIMAL(10,2),

        bidding_strategy VARCHAR(50),

        start_date DATE,
        end_date DATE,

        last_synced_at TIMESTAMP WITH TIME ZONE,
        raw_data JSONB,

        created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
        updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

        UNIQUE(profile_id, amazon_campaign_id)
    );

CREATE INDEX idx_campaigns_profile ON campaigns(profile_id);
CREATE INDEX idx_campaigns_state ON campaigns(state) WHERE state = 'enabled';

CREATE TABLE ad_groups (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    campaign_id UUID NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,

    amazon_ad_group_id BIGINT NOT NULL,
    name VARCHAR(500) NOT NULL,
    state VARCHAR(50) NOT NULL
        CHECK (state IN ('enabled','paused','archived')),

    default_bid DECIMAL(10,4),

    last_synced_at TIMESTAMP WITH TIME ZONE,
    raw_data JSONB,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    UNIQUE(campaign_id, amazon_ad_group_id)
);

CREATE INDEX idx_ad_groups_campaign ON ad_groups(campaign_id);

CREATE TABLE keywords (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ad_group_id UUID NOT NULL REFERENCES ad_groups(id) ON DELETE CASCADE,

```

```

amazon_keyword_id BIGINT NOT NULL,
keyword_text VARCHAR(500) NOT NULL,
match_type VARCHAR(50) NOT NULL
    CHECK (match_type IN ('exact','phrase','broad')),
state VARCHAR(50) NOT NULL
    CHECK (state IN ('enabled','paused','archived')),

bid DECIMAL(10,4),

last_synced_at TIMESTAMP WITH TIME ZONE,
raw_data JSONB,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

UNIQUE(ad_group_id, amazon_keyword_id)
);

CREATE INDEX idx_keywords_ad_group ON keywords(ad_group_id);
CREATE INDEX idx_keywords_text ON keywords(keyword_text);
CREATE INDEX idx_keywords_state ON keywords(state) WHERE state = 'enabled';

CREATE TABLE negative_keywords (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    campaign_id UUID REFERENCES campaigns(id) ON DELETE CASCADE,
    ad_group_id UUID REFERENCES ad_groups(id) ON DELETE CASCADE,

    amazon_negative_keyword_id BIGINT NOT NULL,
    keyword_text VARCHAR(500) NOT NULL,
    match_type VARCHAR(50) NOT NULL
        CHECK (match_type IN ('negativeExact','negativePhrase')),
    state VARCHAR(50) NOT NULL DEFAULT 'enabled',

    last_synced_at TIMESTAMP WITH TIME ZONE,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    CONSTRAINT chk_negative_level CHECK (
        (campaign_id IS NOT NULL AND ad_group_id IS NULL) OR
        (campaign_id IS NULL AND ad_group_id IS NOT NULL)
    )
);

CREATE INDEX idx_negatives_campaign ON negative_keywords(campaign_id) WHERE
campaign_id IS NOT NULL;
CREATE INDEX idx_negatives_ad_group ON negative_keywords(ad_group_id) WHERE
ad_group_id IS NOT NULL;

```

```

CREATE TABLE product_targets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ad_group_id UUID NOT NULL REFERENCES ad_groups(id) ON DELETE CASCADE,

  amazon_target_id BIGINT NOT NULL,
  expression_type VARCHAR(50) NOT NULL,
  expression JSONB NOT NULL,
  state VARCHAR(50) NOT NULL
    CHECK (state IN ('enabled','paused','archived')),

  bid DECIMAL(10,4),

  last_synced_at TIMESTAMP WITH TIME ZONE,
  raw_data JSONB,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

  UNIQUE(ad_group_id, amazon_target_id)
);

```

```

-- =====
-- SEARCH TERMS (FINAL: query_norm + hash + contexte complet)
-- =====

```

```

CREATE TABLE search_terms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  profile_id UUID NOT NULL REFERENCES marketplace_profiles(id) ON DELETE
  CASCADE,
  campaign_id UUID NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,
  ad_group_id UUID NOT NULL REFERENCES ad_groups(id) ON DELETE CASCADE,

  keyword_id UUID REFERENCES keywords(id) ON DELETE SET NULL,
  product_target_id UUID REFERENCES product_targets(id) ON DELETE SET NULL,

  amazon_campaign_id BIGINT,
  amazon_ad_group_id BIGINT,
  amazon_keyword_id BIGINT,
  amazon_target_id BIGINT,

  -- Affichage
  query VARCHAR(500) NOT NULL,

  -- FINAL: normalisation pour dedup robuste
  query_norm VARCHAR(500) GENERATED ALWAYS AS (lower(trim(query))) STORED,

```

```

-- Hash sur query_norm (stable)
query_hash VARCHAR(32) GENERATED ALWAYS AS (md5(query_norm)) STORED,

match_type VARCHAR(50),

targeting_type VARCHAR(50)
    CHECK (targeting_type IN ('keyword','target','auto')),

first_seen_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
last_seen_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

status VARCHAR(50) DEFAULT 'new'
    CHECK (status IN ('new','reviewed','promoted','negated','ignored')),

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

-- Dédup par query_norm dans le même ad_group
UNIQUE(ad_group_id, query_hash)
);

CREATE INDEX idx_search_terms_status ON search_terms(status);
CREATE INDEX idx_search_terms_campaign ON search_terms(campaign_id);
CREATE INDEX idx_search_terms_query ON search_terms(query);
CREATE INDEX idx_search_terms_query_norm ON search_terms(query_norm);

-- =====
-- CAMPAIGN <-> BOOK MAPPING
-- =====

CREATE TABLE campaign_book_mapping (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    book_id UUID NOT NULL REFERENCES books(id) ON DELETE CASCADE,
    campaign_id UUID NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE,

    is_primary BOOLEAN DEFAULT false,
    notes TEXT,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    UNIQUE(book_id, campaign_id)
);

-- =====
-- METRICS (FINAL: entity_key polymorphe, search_term contextuel)
-- =====

CREATE TABLE daily_metrics (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

entity_type VARCHAR(50) NOT NULL
    CHECK (entity_type IN ('campaign','ad_group','keyword','target','search_term')),
entity_key TEXT NOT NULL,

profile_id UUID NOT NULL REFERENCES marketplace_profiles(id) ON DELETE
CASCADE,

date DATE NOT NULL,

marketplace VARCHAR(10) NOT NULL,
currency VARCHAR(3) NOT NULL,

impressions INTEGER DEFAULT 0,
clicks INTEGER DEFAULT 0,
spend DECIMAL(12,4) DEFAULT 0,
sales DECIMAL(12,4) DEFAULT 0,
orders INTEGER DEFAULT 0,
units INTEGER DEFAULT 0,

attribution_window VARCHAR(10) DEFAULT '7d',

synced_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

UNIQUE(entity_type, entity_key, date)
);

CREATE INDEX idx_metrics_entity ON daily_metrics(entity_type, entity_key);
CREATE INDEX idx_metrics_date ON daily_metrics(date DESC);
CREATE INDEX idx_metrics_profile_date ON daily_metrics(profile_id, date DESC);

-- =====
-- REPORT JOBS (FINAL: statuts prêts/téléchargés/ingérés)
-- =====

CREATE TABLE report_jobs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    profile_id UUID NOT NULL REFERENCES marketplace_profiles(id) ON DELETE
CASCADE,

report_type VARCHAR(50) NOT NULL
    CHECK (report_type IN ('campaigns','ad_groups','keywords','targets','search_terms')),
amazon_report_id VARCHAR(200),

date_from DATE NOT NULL,
date_to DATE NOT NULL,

```

```

-- FINAL: workflow clair
status VARCHAR(50) NOT NULL DEFAULT 'pending'
CHECK (status IN (
    'pending','requested','processing',
    'ready','downloaded','ingested',
    'failed'
)),

requested_at TIMESTAMP WITH TIME ZONE,
completed_at TIMESTAMP WITH TIME ZONE, -- "ready" chez Amazon
downloaded_at TIMESTAMP WITH TIME ZONE,
ingested_at TIMESTAMP WITH TIME ZONE,

download_url TEXT,

records_processed INTEGER DEFAULT 0,
error_message TEXT,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

UNIQUE(profile_id, report_type, date_from, date_to)
);

CREATE INDEX idx_report_jobs_status
ON report_jobs(status)
WHERE status NOT IN ('ingested');

-- =====
-- RULES
-- =====

CREATE TABLE rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,

    name VARCHAR(255) NOT NULL,
    description TEXT,
    rule_type VARCHAR(50) NOT NULL
    CHECK (rule_type IN (
        'bid_adjustment','pause_keyword','enable_keyword',
        'harvest_search_term','add_negative',
        'budget_alert','acos_alert'
    )),

    applies_to VARCHAR(50) NOT NULL
    CHECK (applies_to IN ('campaign','ad_group','keyword','target','search_term')),

    conditions JSONB NOT NULL,

```

```

actions JSONB NOT NULL,

mode VARCHAR(50) DEFAULT 'recommend'
  CHECK (mode IN ('recommend','auto')),
priority INTEGER DEFAULT 100,

max_daily_executions INTEGER DEFAULT 10,
cooldown_hours INTEGER DEFAULT 48,

is_active BOOLEAN DEFAULT true,

version INTEGER DEFAULT 1,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_rules_workspace ON rules(workspace_id);
CREATE INDEX idx_rules_active ON rules(is_active, priority) WHERE is_active = true;

-- =====
-- RECOMMENDATIONS (rule_snapshot + entity_key)
-- =====

CREATE TABLE recommendations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  rule_id UUID REFERENCES rules(id) ON DELETE SET NULL,

  entity_type VARCHAR(50) NOT NULL,
  entity_key TEXT NOT NULL,

  action_type VARCHAR(50) NOT NULL,
  suggested_action JSONB NOT NULL,

  context_data JSONB,
  confidence_score DECIMAL(5,4),

  rule_snapshot JSONB,

  status VARCHAR(50) DEFAULT 'pending'
  CHECK (status IN ('pending','approved','rejected','executed','expired','skipped')),

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  expires_at TIMESTAMP WITH TIME ZONE DEFAULT (NOW() + INTERVAL '7 days'),
  reviewed_at TIMESTAMP WITH TIME ZONE,
  reviewed_by VARCHAR(50),

```

```

    action_id UUID
);

CREATE INDEX idx_reco_status ON recommendations(status, workspace_id);
CREATE INDEX idx_reco_entity ON recommendations(entity_type, entity_key);
CREATE INDEX idx_reco_pending ON recommendations(status, created_at) WHERE status
= 'pending';

-- =====
-- ACTION LOG (entity_key)
-- =====

CREATE TABLE action_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,

    recommendation_id UUID REFERENCES recommendations(id),
    rule_id UUID REFERENCES rules(id),

    entity_type VARCHAR(50) NOT NULL,
    entity_key TEXT NOT NULL,
    amazon_entity_id BIGINT,

    action_type VARCHAR(50) NOT NULL,

    before_value JSONB,
    after_value JSONB,
    rationale TEXT NOT NULL,

    executed_by VARCHAR(50) NOT NULL
        CHECK (executed_by IN ('system','user','manual')),
    executed_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    status VARCHAR(50) NOT NULL
        CHECK (status IN ('pending','success','failed','rolled_back','simulated')),

    api_request JSONB,
    api_response JSONB,
    error_message TEXT,

    is_reversible BOOLEAN DEFAULT false,
    rolled_back_at TIMESTAMP WITH TIME ZONE,
    rollback_action_id UUID REFERENCES action_log(id),

    dry_run BOOLEAN DEFAULT false,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

```

```

CREATE INDEX idx_action_workspace ON action_log(workspace_id);
CREATE INDEX idx_action_entity ON action_log(entity_type, entity_key);
CREATE INDEX idx_action_date ON action_log(executed_at DESC);
CREATE INDEX idx_action_status ON action_log(status);

-- =====
-- OBSERVABILITY
-- =====

CREATE TABLE sync_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ad_account_id UUID NOT NULL REFERENCES ad_accounts(id) ON DELETE
  CASCADE,
  profile_id UUID REFERENCES marketplace_profiles(id) ON DELETE CASCADE,

  job_name VARCHAR(100) NOT NULL,
  job_type VARCHAR(50) NOT NULL,

  started_at TIMESTAMP WITH TIME ZONE NOT NULL,
  finished_at TIMESTAMP WITH TIME ZONE,
  duration_seconds INTEGER,

  status VARCHAR(50) NOT NULL
    CHECK (status IN ('running','success','partial','failed')),

  records_fetched INTEGER DEFAULT 0,
  records_created INTEGER DEFAULT 0,
  records_updated INTEGER DEFAULT 0,
  records_failed INTEGER DEFAULT 0,

  details JSONB,
  errors JSONB,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_sync_account ON sync_logs(ad_account_id);
CREATE INDEX idx_sync_date ON sync_logs(started_at DESC);

CREATE TABLE incidents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID REFERENCES workspaces(id) ON DELETE CASCADE,

  incident_type VARCHAR(100) NOT NULL,
  severity VARCHAR(50) NOT NULL
    CHECK (severity IN ('info','warning','error','critical')),

```

```

title VARCHAR(500) NOT NULL,
description TEXT,
context JSONB,

entity_type VARCHAR(50),
entity_key TEXT,

status VARCHAR(50) DEFAULT 'open'
    CHECK (status IN ('open','acknowledged','resolved','ignored')),

occurred_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
acknowledged_at TIMESTAMP WITH TIME ZONE,
resolved_at TIMESTAMP WITH TIME ZONE,

notification_sent BOOLEAN DEFAULT false,
notification_channel VARCHAR(50),
notification_sent_at TIMESTAMP WITH TIME ZONE,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_incidents_status ON incidents(status, severity);
CREATE INDEX idx_incidents_workspace ON incidents(workspace_id);

-- =====
-- SYSTEM CONFIG
-- =====

CREATE TABLE system_config (
    key VARCHAR(100) PRIMARY KEY,
    value JSONB NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

INSERT INTO system_config (key, value) VALUES
    ('kill_switch', '{"enabled": false, "reason": null, "enabled_at": null, "enabled_by": null}'),
    ('global_settings', '{"dry_run": true, "max_actions_per_day": 50}');

```

3.2 Migration 002 — Vues & Fonctions (FINAL)

```

-- =====
-- MIGRATION 002: Views & Functions (FINAL)
-- =====

-- =====
-- VUE: Métriques + KPIs calculés

```

-- =====

```
CREATE OR REPLACE VIEW v_daily_metrics_with_kpi AS
SELECT
  dm.*,
  CASE WHEN dm.sales > 0
    THEN ROUND((dm.spend / dm.sales * 100)::numeric, 2)
    ELSE NULL END AS acos,
  CASE WHEN dm.spend > 0
    THEN ROUND((dm.sales / dm.spend)::numeric, 2)
    ELSE NULL END AS roas,
  CASE WHEN dm.impressions > 0
    THEN ROUND((dm.clicks::numeric / dm.impressions * 100), 4)
    ELSE NULL END AS ctr,
  CASE WHEN dm.clicks > 0
    THEN ROUND((dm.orders::numeric / dm.clicks * 100), 4)
    ELSE NULL END AS cvr,
  CASE WHEN dm.clicks > 0
    THEN ROUND((dm.spend / dm.clicks)::numeric, 4)
    ELSE NULL END AS cpc
FROM daily_metrics dm;
```

-- =====

-- VUE: Performance par livre

-- =====

```
CREATE OR REPLACE VIEW v_book_performance AS
SELECT
  b.id AS book_id,
  b.asin,
  b.title,
  b.marketplace,
  dm.date,
  SUM(dm.impressions) AS impressions,
  SUM(dm.clicks) AS clicks,
  SUM(dm.spend) AS spend,
  SUM(dm.sales) AS sales,
  SUM(dm.orders) AS orders,
  CASE WHEN SUM(dm.sales) > 0
    THEN ROUND((SUM(dm.spend) / SUM(dm.sales) * 100)::numeric, 2)
    ELSE NULL END AS acos
FROM books b
JOIN campaign_book_mapping cbm ON cbm.book_id = b.id
JOIN campaigns c ON c.id = cbm.campaign_id
JOIN daily_metrics dm
  ON dm.entity_type = 'campaign'
 AND dm.entity_key = 'campaign:' || c.amazon_campaign_id
GROUP BY b.id, b.asin, b.title, b.marketplace, dm.date;
```

```
-- =====
-- VUE: Keywords nécessitant attention (clics sans ventes)
-- =====
```

```
CREATE OR REPLACE VIEW v_keywords_needing_attention AS
WITH keyword_metrics AS (
    SELECT
        k.id,
        k.keyword_text,
        k.match_type,
        k.bid,
        k.state,
        ag.name AS ad_group_name,
        c.name AS campaign_name,
        c.id AS campaign_id,
        SUM(dm.clicks) AS clicks_7d,
        SUM(dm.orders) AS orders_7d,
        SUM(dm.spend) AS spend_7d,
        SUM(dm.sales) AS sales_7d
    FROM keywords k
    JOIN ad_groups ag ON ag.id = k.ad_group_id
    JOIN campaigns c ON c.id = ag.campaign_id
    LEFT JOIN daily_metrics dm
        ON dm.entity_type = 'keyword'
        AND dm.entity_key = 'keyword:' || k.amazon_keyword_id
        AND dm.date >= CURRENT_DATE - INTERVAL '7 days'
    WHERE k.state = 'enabled'
    GROUP BY k.id, k.keyword_text, k.match_type, k.bid, k.state, ag.name, c.name, c.id
)
SELECT *,
    CASE WHEN sales_7d > 0
        THEN ROUND((spend_7d / sales_7d * 100)::numeric, 2)
        ELSE NULL END AS acos_7d
FROM keyword_metrics
WHERE clicks_7d >= 15 AND orders_7d = 0;
```

```
-- =====
-- VUE: Search terms à promouvoir (FINAL: entity_key contextuel)
-- =====
```

```
CREATE OR REPLACE VIEW v_search_terms_to_promote AS
WITH st_metrics AS (
    SELECT
        st.id,
        st.query,
        st.query_hash,
        st.amazon_ad_group_id,
```

```

        st.status,
        st.campaign_id,
        st.ad_group_id,
        st.keyword_id,
        k.keyword_text AS source_keyword,
        SUM(dm.clicks) AS clicks_30d,
        SUM(dm.orders) AS orders_30d,
        SUM(dm.sales) AS sales_30d,
        SUM(dm.spend) AS spend_30d
    FROM search_terms st
    LEFT JOIN keywords k ON k.id = st.keyword_id
    LEFT JOIN daily_metrics dm
        ON dm.entity_type = 'search_term'
    AND dm.entity_key = ('search_term:' || st.amazon_ad_group_id || ':' || st.query_hash)
    AND dm.date >= CURRENT_DATE - INTERVAL '30 days'
    WHERE st.status = 'new'
    GROUP BY st.id, st.query, st.query_hash, st.amazon_ad_group_id, st.status,
        st.campaign_id, st.ad_group_id, st.keyword_id, k.keyword_text
)
SELECT *,
    CASE WHEN sales_30d > 0
        THEN ROUND((spend_30d / sales_30d * 100)::numeric, 2)
        ELSE NULL END AS acos_30d
FROM st_metrics
WHERE orders_30d >= 2;

```

```

-- =====
-- VUE: Search terms à bloquer (FINAL: entity_key contextuel)
-- =====

```

```

CREATE OR REPLACE VIEW v_search_terms_to_negate AS
WITH st_metrics AS (
    SELECT
        st.id,
        st.query,
        st.query_hash,
        st.amazon_ad_group_id,
        st.status,
        st.campaign_id,
        st.ad_group_id,
        SUM(dm.clicks) AS clicks_30d,
        SUM(dm.orders) AS orders_30d,
        SUM(dm.spend) AS spend_30d
    FROM search_terms st
    LEFT JOIN daily_metrics dm
        ON dm.entity_type = 'search_term'
    AND dm.entity_key = ('search_term:' || st.amazon_ad_group_id || ':' || st.query_hash)
    AND dm.date >= CURRENT_DATE - INTERVAL '30 days'

```

```

WHERE st.status = 'new'
GROUP BY st.id, st.query, st.query_hash, st.amazon_ad_group_id, st.status,
        st.campaign_id, st.ad_group_id
)
SELECT *
FROM st_metrics
WHERE clicks_30d >= 15 AND orders_30d = 0;

-- =====
-- VUE: Résumé quotidien
-- =====

CREATE OR REPLACE VIEW v_daily_summary AS
SELECT
    mp.ad_account_id,
    aa.workspace_id,
    dm.date,
    dm.marketplace,
    SUM(dm.impressions) AS total_impressions,
    SUM(dm.clicks) AS total_clicks,
    SUM(dm.spend) AS total_spend,
    SUM(dm.sales) AS total_sales,
    SUM(dm.orders) AS total_orders,
    CASE WHEN SUM(dm.sales) > 0
        THEN ROUND((SUM(dm.spend) / SUM(dm.sales) * 100)::numeric, 2)
        ELSE NULL END AS overall_acos,
    COUNT(DISTINCT CASE WHEN dm.entity_type = 'campaign' THEN dm.entity_key END)
AS active_campaigns
FROM daily_metrics dm
JOIN marketplace_profiles mp ON mp.id = dm.profile_id
JOIN ad_accounts aa ON aa.id = mp.ad_account_id
WHERE dm.entity_type = 'campaign'
GROUP BY mp.ad_account_id, aa.workspace_id, dm.date, dm.marketplace;

-- =====
-- Fonctions utilitaires
-- =====

CREATE OR REPLACE FUNCTION make_entity_key(p_type TEXT, p_amazon_id BIGINT)
RETURNS TEXT AS $$
BEGIN
    RETURN p_type || ':' || p_amazon_id::TEXT;
END;
$$ LANGUAGE plpgsql IMMUTABLE;

CREATE OR REPLACE FUNCTION extract_amazon_id(p_entity_key TEXT)
RETURNS BIGINT AS $$
BEGIN

```

```
    RETURN split_part(p_entity_key, ':', 2)::BIGINT;
EXCEPTION WHEN OTHERS THEN
    RETURN NULL;
END;
$$ LANGUAGE plpgsql IMMUTABLE;
```

```
CREATE OR REPLACE FUNCTION is_in_cooldown(
    p_entity_key TEXT,
    p_rule_id UUID,
    p_cooldown_hours INTEGER
)
RETURNS BOOLEAN AS $$
DECLARE
    v_last_action TIMESTAMP WITH TIME ZONE;
BEGIN
    SELECT MAX(executed_at) INTO v_last_action
    FROM action_log
    WHERE entity_key = p_entity_key
        AND rule_id = p_rule_id
        AND status = 'success';

    IF v_last_action IS NULL THEN
        RETURN FALSE;
    END IF;

    RETURN v_last_action > (NOW() - (p_cooldown_hours || ' hours')::INTERVAL);
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION count_daily_rule_executions(p_rule_id UUID)
RETURNS INTEGER AS $$
BEGIN
    RETURN (
        SELECT COUNT(*)
        FROM action_log
        WHERE rule_id = p_rule_id
            AND executed_at >= CURRENT_DATE
            AND status = 'success'
    );
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE FUNCTION is_kill_switch_active()
RETURNS BOOLEAN AS $$
BEGIN
    RETURN (
        SELECT (value->>'enabled')::BOOLEAN
        FROM system_config
    );
END;
```

```

        WHERE key = 'kill_switch'
    );
END;
$$ LANGUAGE plpgsql;

```

3.3 Migration 003 — Règles par défaut (INCHANGÉE)

Identique à la version Claude (7 règles).

Note importante d'implémentation : remplacer le workspace_id placeholder via [seed-workspace.ts](#).

(Je te remets la même migration 003 que Claude, inchangée, pour garder ton doc cohérent.)

```

-- =====
-- MIGRATION 003: Default Rules
-- =====

-- Note: workspace_id sera remplacé par le vrai UUID après création du workspace

-- Règle 1: Baisse enchère - Clics sans vente
INSERT INTO rules (workspace_id, name, description, rule_type, applies_to, conditions,
actions, mode, priority, cooldown_hours, max_daily_executions)
VALUES (
    '00000000-0000-0000-0000-000000000000',
    'bid_down_no_sales',
    'Baisser l"enchère des keywords avec 15+ clics et 0 vente sur 7 jours',
    'bid_adjustment',
    'keyword',
    {
        "operator": "AND",
        "conditions": [
            {"metric": "clicks", "operator": ">=", "value": 15, "period_days": 7},
            {"metric": "orders", "operator": "=", "value": 0, "period_days": 7}
        ]
    },
    {
        "action_type": "adjust_bid",
        "adjustment_type": "percentage",
        "adjustment_value": -20,
        "min_value": 0.10,
        "max_value": null
    },
    'recommend',
    10,
    72,
    10

```

```
);
```

-- Règle 2: Hausse enchère - Performance excellente

```
INSERT INTO rules (workspace_id, name, description, rule_type, applies_to, conditions,
actions, mode, priority, cooldown_hours, max_daily_executions)
```

```
VALUES (
```

```
  '00000000-0000-0000-0000-000000000000',
```

```
  'bid_up_high_performer',
```

```
  'Augmenter l'enchère des keywords avec ACOS < 25% et 3+ ventes',
```

```
  'bid_adjustment',
```

```
  'keyword',
```

```
  '{
```

```
    "operator": "AND",
```

```
    "conditions": [
```

```
      {"metric": "orders", "operator": ">=", "value": 3, "period_days": 14},
```

```
      {"metric": "acos", "operator": "<", "value": 25, "period_days": 14},
```

```
      {"metric": "clicks", "operator": ">=", "value": 20, "period_days": 14}
```

```
    ]
```

```
  },
```

```
  '{
```

```
    "action_type": "adjust_bid",
```

```
    "adjustment_type": "percentage",
```

```
    "adjustment_value": 15,
```

```
    "min_value": null,
```

```
    "max_value": 2.00
```

```
  },
```

```
  'recommend',
```

```
  20,
```

```
  168,
```

```
  5
```

```
);
```

-- Règle 3: Pause - ACOS catastrophique

```
INSERT INTO rules (workspace_id, name, description, rule_type, applies_to, conditions,
actions, mode, priority, cooldown_hours, max_daily_executions)
```

```
VALUES (
```

```
  '00000000-0000-0000-0000-000000000000',
```

```
  'pause_high_acos',
```

```
  'Mettre en pause les keywords avec ACOS > 100% et 10€+ dépensés',
```

```
  'pause_keyword',
```

```
  'keyword',
```

```
  '{
```

```
    "operator": "AND",
```

```
    "conditions": [
```

```
      {"metric": "spend", "operator": ">=", "value": 10, "period_days": 14},
```

```
      {"metric": "acos", "operator": ">", "value": 100, "period_days": 14}
```

```
    ]
```

```
  },
```

```
{
  "action_type": "pause"
},
'recommend',
5,
0,
20
);
```

-- Règle 4: Harvest - Search term rentable

INSERT INTO rules (workspace_id, name, description, rule_type, applies_to, conditions, actions, mode, priority, cooldown_hours, max_daily_executions)

```
VALUES (
  '00000000-0000-0000-0000-000000000000',
  'harvest_profitable_search_term',
  'Promouvoir en keyword exact les search terms avec 2+ ventes et ACOS < 40%',
  'harvest_search_term',
  'search_term',
  '{
    "operator": "AND",
    "conditions": [
      {"metric": "orders", "operator": ">=", "value": 2, "period_days": 30},
      {"metric": "acos", "operator": "<", "value": 40, "period_days": 30}
    ]
  }',
  '{
    "action_type": "create_keyword",
    "match_type": "exact",
    "bid_strategy": "copy_from_parent"
  }',
  'recommend',
  30,
  0,
  10
);
```

-- Règle 5: Negative - Search term non rentable

INSERT INTO rules (workspace_id, name, description, rule_type, applies_to, conditions, actions, mode, priority, cooldown_hours, max_daily_executions)

```
VALUES (
  '00000000-0000-0000-0000-000000000000',
  'negative_unprofitable_search_term',
  'Ajouter en negative les search terms avec 15+ clics et 0 vente',
  'add_negative',
  'search_term',
  '{
    "operator": "AND",
    "conditions": [
```

```

        {"metric": "clicks", "operator": ">=", "value": 15, "period_days": 30},
        {"metric": "orders", "operator": "=", "value": 0, "period_days": 30}
    ]
}',
'{
    "action_type": "create_negative",
    "match_type": "negativeExact",
    "level": "ad_group"
}',
'recommend',
15,
0,
15
);

```

-- Règle 6: Alerte - Spike ACOS

INSERT INTO rules (workspace_id, name, description, rule_type, applies_to, conditions, actions, mode, priority, cooldown_hours, max_daily_executions)

```

VALUES (
    '00000000-0000-0000-0000-000000000000',
    'alert_acos_spike',
    'Alerter si ACOS augmente de 50%+ vs semaine précédente',
    'acos_alert',
    'campaign',
    '{
        "operator": "AND",
        "conditions": [
            {"metric": "spend", "operator": ">=", "value": 20, "period_days": 7},
            {"metric": "acos_change_pct", "operator": ">", "value": 50, "period_days": 7}
        ]
    }',
    '{
        "action_type": "alert",
        "severity": "warning"
    }',
    'auto',
    8,
    48,
    5
);

```

-- Règle 7: Alerte - Dépense sans vente

INSERT INTO rules (workspace_id, name, description, rule_type, applies_to, conditions, actions, mode, priority, cooldown_hours, max_daily_executions)

```

VALUES (
    '00000000-0000-0000-0000-000000000000',
    'alert_spend_no_sales',
    'Alerter si 20€+ dépensés sur 3 jours sans aucune vente',

```

```

'acos_alert',
'campaign',
{
  "operator": "AND",
  "conditions": [
    {"metric": "spend", "operator": ">=", "value": 20, "period_days": 3},
    {"metric": "orders", "operator": "=", "value": 0, "period_days": 3}
  ]
},
{
  "action_type": "alert",
  "severity": "error"
},
'auto',
7,
24,
10
);

```

4. Endpoints API V1

4.1 Liste des Endpoints

Méthode	Endpoint	Description
Auth		
GET	/auth/amazon	Initier OAuth Amazon
GET	/auth/amazon/callback	Callback OAuth
POST	/auth/refresh	Refresh token manuel
Sync		
POST	/api/sync/trigger	Déclencher sync manuel
GET	/api/sync/status	Status du dernier sync
Reports		
POST	/api/reports/request	Demander un report
GET	/api/reports/:id/status	Poll status report
Rules		

GET	<code>/api/rules</code>	Lister les règles
POST	<code>/api/rules/evaluate</code>	Déclencher évaluation
PATCH	<code>/api/rules/:id</code>	Modifier une règle
Recommendations		
GET	<code>/api/recommendations</code>	Lister (filtrable par status)
POST	<code>/api/recommendations/:id/approve</code>	Approuver
POST	<code>/api/recommendations/:id/reject</code>	Rejeter
Actions		
POST	<code>/api/actions/execute</code>	Exécuter actions approuvées
GET	<code>/api/actions/log</code>	Historique des actions
Books		
GET	<code>/api/books</code>	Lister les livres
POST	<code>/api/books</code>	Ajouter un livre
PATCH	<code>/api/books/:id</code>	Modifier (ACOS target, etc.)
Metrics		
GET	<code>/api/metrics/summary</code>	Résumé quotidien
GET	<code>/api/metrics/entity/:type/:key</code>	Métriques d'une entité
System		
GET	<code>/api/health</code>	Health check
GET	<code>/api/system/config</code>	Lire config
POST	<code>/api/system/kill-switch</code>	Toggle kill switch

4.2 Payloads JSON

POST /api/sync/trigger

```
{
  "ad_account_id": "uuid",
  "profile_ids": ["uuid"],
  "sync_type": "full"
}
```

POST /api/reports/request

```
{
  "profile_id": "uuid",
  "report_type": "keywords",
  "date_from": "2026-01-01",
  "date_to": "2026-01-31"
}
```

POST /api/rules/evaluate

```
{
  "workspace_id": "uuid",
  "rule_ids": ["uuid"],
  "dry_run": true
}
```

POST /api/recommendations/:id/approve

```
{
  "comment": "OK, validé"
}
```

POST /api/actions/execute

```
{
  "recommendation_ids": ["uuid", "uuid"],
  "dry_run": false
}
```

POST /api/system/kill-switch

```
{
  "enabled": true,
  "reason": "Investigation anomalie ACOS"
}
```

5. Workflows n8n V1 (Minimaux)

(Identiques à la version Claude, car compatibles avec les ajustements DB.)

5.1 Workflow 1: Nightly Sync

```
{
  "name": "01-Nightly-Sync",
  "trigger": "CRON 0 3 * * *",
  "nodes": [
    {
      "type": "cron", "expression": "0 3 * * *",
      {
        "type": "http_request",
        "method": "POST",
        "url": "{{API_URL}}/api/sync/trigger",
        "body": { "sync_type": "full" }
      },
      {
        "type": "if", "condition": "{{json.status}} != 'success'",
        {
          "type": "telegram", "message": "❌ Sync failed: {{json.error}}"
        }
      }
    ]
  }
}
```

5.2 Workflow 2: Evaluate Rules

```
{
  "name": "02-Evaluate-Rules",
  "trigger": "CRON 0 5 * * *",
  "nodes": [
    {
      "type": "cron", "expression": "0 5 * * *",
      {
        "type": "http_request", "method": "GET", "url": "{{API_URL}}/api/sync/status",
        {
          "type": "if", "condition": "{{json.last_sync_status}} == 'success'",
          {
            "type": "http_request",
            "method": "POST",
            "url": "{{API_URL}}/api/rules/evaluate",
            "body": { "dry_run": false }
          },
          {
            "type": "if", "condition": "{{json.recommendations_created}} > 0",
            {
              "type": "telegram",
              "message": "📋 {{json.recommendations_created}} nouvelles  
recommandations\n\n{{formatRecommendations(json.recommendations)}}"
            }
          }
        }
      }
    ]
  }
}
```

5.3 Workflow 3: Execute Approved

```
{
```

```

"name": "03-Execute-Approved",
"trigger": "CRON 0 6,18 * * *",
"nodes": [
  {"type": "cron", "expression": "0 6,18 * * *"},
  {"type": "http_request", "method": "GET", "url":
"{{API_URL}}/api/recommendations?status=approved"},
  {"type": "if", "condition": "{{$.length}} > 0"},
  {
    "type": "http_request",
    "method": "POST",
    "url": "{{API_URL}}/api/actions/execute",
    "body": { "dry_run": "{{DRY_RUN}}" }
  },
  {"type": "telegram", "message": "✅ {{$.executed}} actions exécutées\n❌
{{$.failed}} échecs"}
]
}

```

5.4 Workflow 4: Daily Summary

```

{
  "name": "04-Daily-Summary",
  "trigger": "CRON 0 8 * * *",
  "nodes": [
    {"type": "cron", "expression": "0 8 * * *"},
    {"type": "http_request", "method": "GET", "url":
"{{API_URL}}/api/metrics/summary?date=yesterday"},
    {
      "type": "telegram",
      "message": "📊 *Rapport Quotidien*\n\n💰 Dépenses: {{$.total_spend}}€\n📈
Ventes: {{$.total_sales}}€ ({{$.total_orders}} commandes)\n📉 ACOS:
{{$.overall_acos}}%\n\n📋 Actions en attente: {{$.pending_recommendations}}"
    }
  ]
}

```

6. Garde-fous V1

6.1 Constantes

```

export const GUARDS = {
  MIN_BID: 0.10,
  MAX_BID: 2.00,
  MAX_BID_INCREASE_PCT: 20,
  MAX_BID_DECREASE_PCT: 30,

```

```

MIN_CLICKS_FOR_DECISION: 15,
MIN_SPEND_FOR_DECISION: 5.0,

MAX_ACTIONS_PER_DAY: 50,
MAX_ACTIONS_PER_RULE_PER_DAY: 10,

DEFAULT_COOLDOWN_HOURS: 48,

REPORT_POLL_TIMEOUT_MINUTES: 30,
REPORT_POLL_INTERVAL_SECONDS: 30,

SEARCH_TERMS_MAX_DAYS_BACK: 60
} as const;

```

6.2 Validation avant exécution

```

function validateBidChange(current: number, proposed: number) {
  const errors: string[] = [];
  const changePct = ((proposed - current) / current) * 100;

  if (proposed < GUARDS.MIN_BID) errors.push(`Bid ${proposed} < min
${GUARDS.MIN_BID}`);
  if (proposed > GUARDS.MAX_BID) errors.push(`Bid ${proposed} > max
${GUARDS.MAX_BID}`);
  if (changePct > GUARDS.MAX_BID_INCREASE_PCT)
    errors.push(`Increase ${changePct.toFixed(1)}% > max
${GUARDS.MAX_BID_INCREASE_PCT}%`);
  if (changePct < -GUARDS.MAX_BID_DECREASE_PCT)
    errors.push(`Decrease ${Math.abs(changePct).toFixed(1)}% > max
${GUARDS.MAX_BID_DECREASE_PCT}%`);

  return {
    valid: errors.length === 0,
    errors,
    adjustedValue: Math.max(GUARDS.MIN_BID, Math.min(GUARDS.MAX_BID, proposed))
  };
}

```

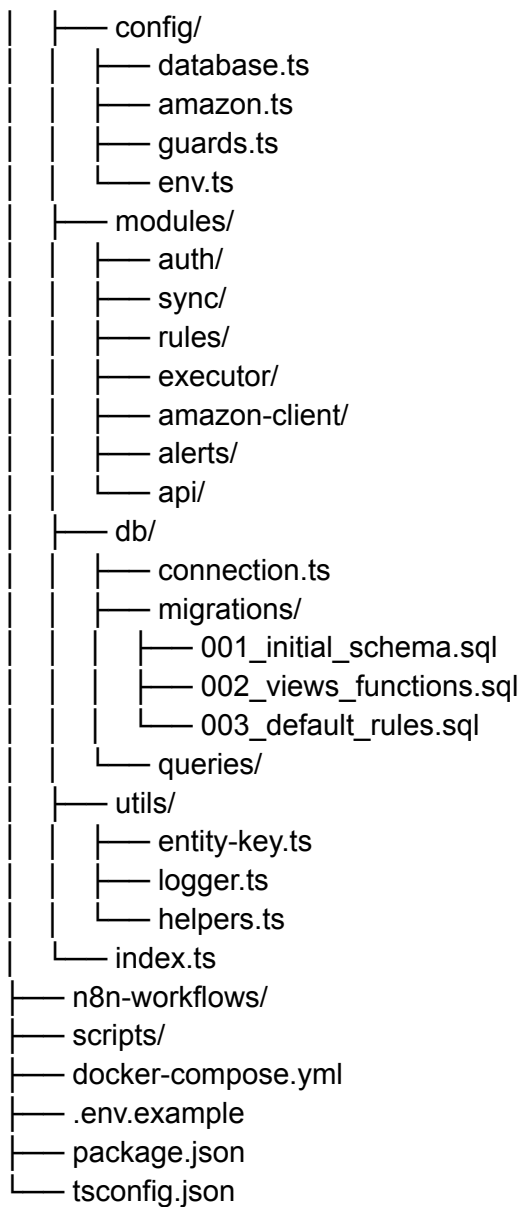
7. Structure du Code V1

(Identique Claude — compatible.)

```

amazon-ads-bot/
├── src/

```



8. Checklist de Lancement V1

Pré-requis

- *Compte Amazon Ads avec accès API*
- *Client ID + Secret LWA obtenus*
- *VPS Hostinger avec Docker*

- PostgreSQL installé
- n8n fonctionnel
- Bot Telegram créé

Phase 1: Setup (Jours 1-3)

- Cloner le repo
- Configurer .env
- Lancer PostgreSQL
- Exécuter migrations
- Créer workspace + user

Phase 2: Auth (Jours 4-7)

- Implémenter OAuth flow
- Tester connexion Amazon
- Stocker refresh token
- Récupérer profiles

Phase 3: Sync (Jours 8-14)

- Sync campaigns
- Sync ad_groups
- Sync keywords
- Sync reports (polling)
- Tester nightly sync

Phase 4: Rules (Jours 15-21)

- Policy engine
- 7 règles par défaut
- Génération recommendations
- Alertes Telegram

Phase 5: Execution (Jours 22-28)

- Executor avec validation
- Action log
- Dry run test
- Kill switch test

Phase 6: Production (Jours 29-35)

- DRY_RUN=false
- Mapper 20 livres
- Monitoring 7 jours
- Ajuster seuils si nécessaire

9. Variables d'Environnement

.env.example

===== APPLICATION =====

NODE_ENV=production

PORT=3001

LOG_LEVEL=info

===== DATABASE =====

DATABASE_URL=postgresql://postgres:password@localhost:5432/amazon_ads_bot

===== SECURITY =====

ENCRYPTION_KEY=your-32-byte-hex-key # openssl rand -hex 32

===== AMAZON ADS =====

AMAZON_CLIENT_ID=amzn1.application-oa2-client.xxxxxx

AMAZON_CLIENT_SECRET=your-secret

AMAZON_REDIRECT_URI=https://yourdomain.com/auth/amazon/callback

AMAZON_ADS_API_BASE_URL=https://advertising-api-eu.amazon.com

===== TELEGRAM =====

TELEGRAM_BOT_TOKEN=123456789:ABC-xxxxx

TELEGRAM_CHAT_ID=your-chat-id

```
# ===== FEATURE FLAGS =====
```

```
DRY_RUN=true
```

```
KILL_SWITCH_ENABLED=false
```

```
# ===== N8N =====
```

```
N8N_WEBHOOK_BASE_URL=http://localhost:5678
```

Note d'implémentation critique (pour éviter les bugs)

Calcul de **entity_key** pour **search_term** métriques (RÈGLE FINALE)

Quand tu upsert les metrics **search_term**, utilise **toujours** :

- **entity_type** = 'search_term'
- **entity_key** = 'search_term:' + **amazon_ad_group_id** + ':' + **query_hash**

Et **query_hash** doit être le hash du **query_norm** (comme dans la table **search_terms**).

Document V1 FINAL — Figé pour implémentation
1er Février 2026 — Version 1.2

Si tu veux, je peux maintenant te livrer **les fichiers prêts à coller** (sans changer le fond) :

- **001_initial_schema.sql**
- **002_views_functions.sql**
- **003_default_rules.sql**

...exactement dans le format dossier **src/db/migrations/**, avec numérotation, commentaires et tout.